



Music Playlist Manager

Presentazione First Sprint



Gruppo 01

0622702428 – Domenico Nappo

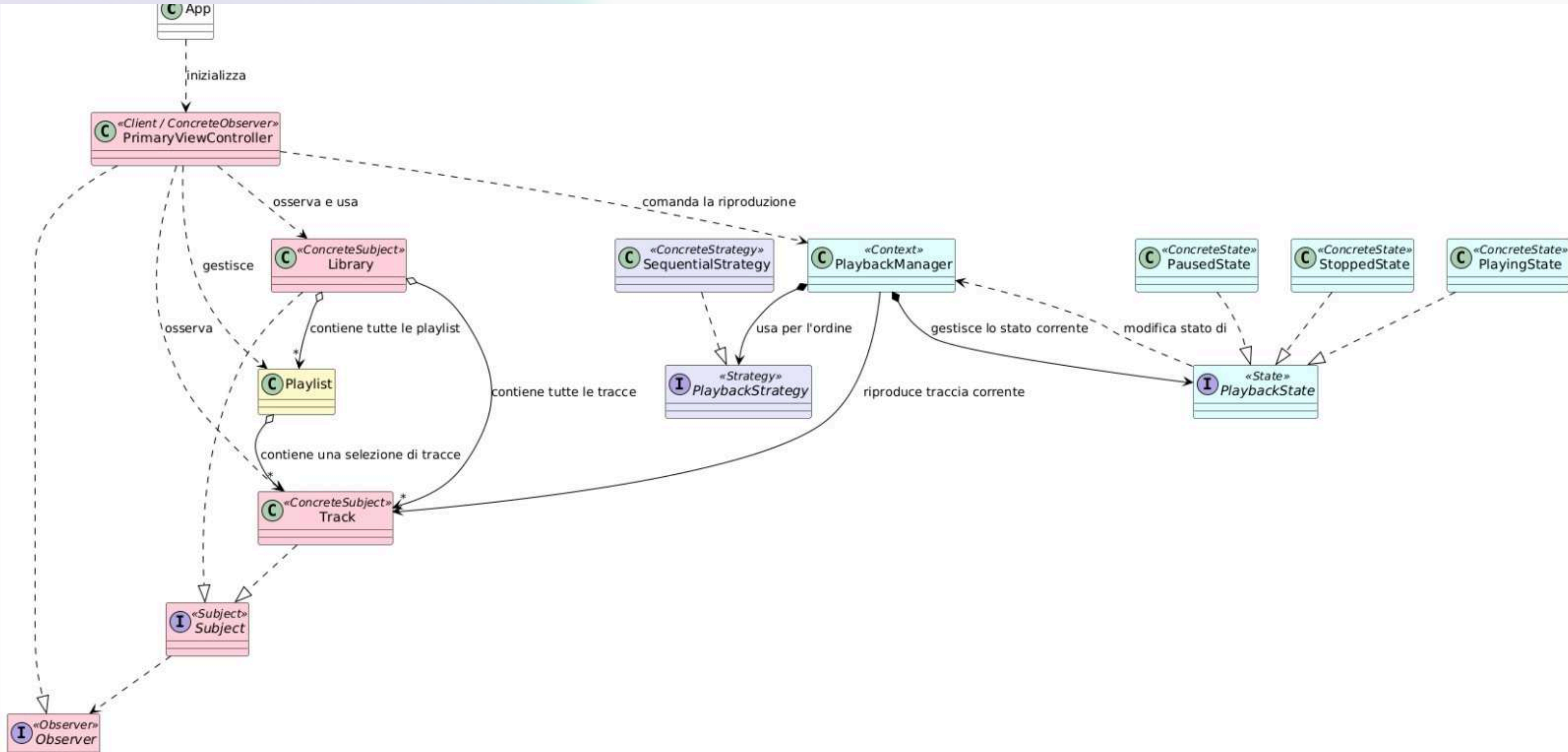
IE22700035 – Arianna Paletta

IE22700189 – Vittorio Postiglione

IE22700199 – Mattia Sanzari

IE22700027 – Debora Villano

Diagramma delle classi



Pattern State - Gestione della riproduzione

CODICE PULITO E LEGGIBILE

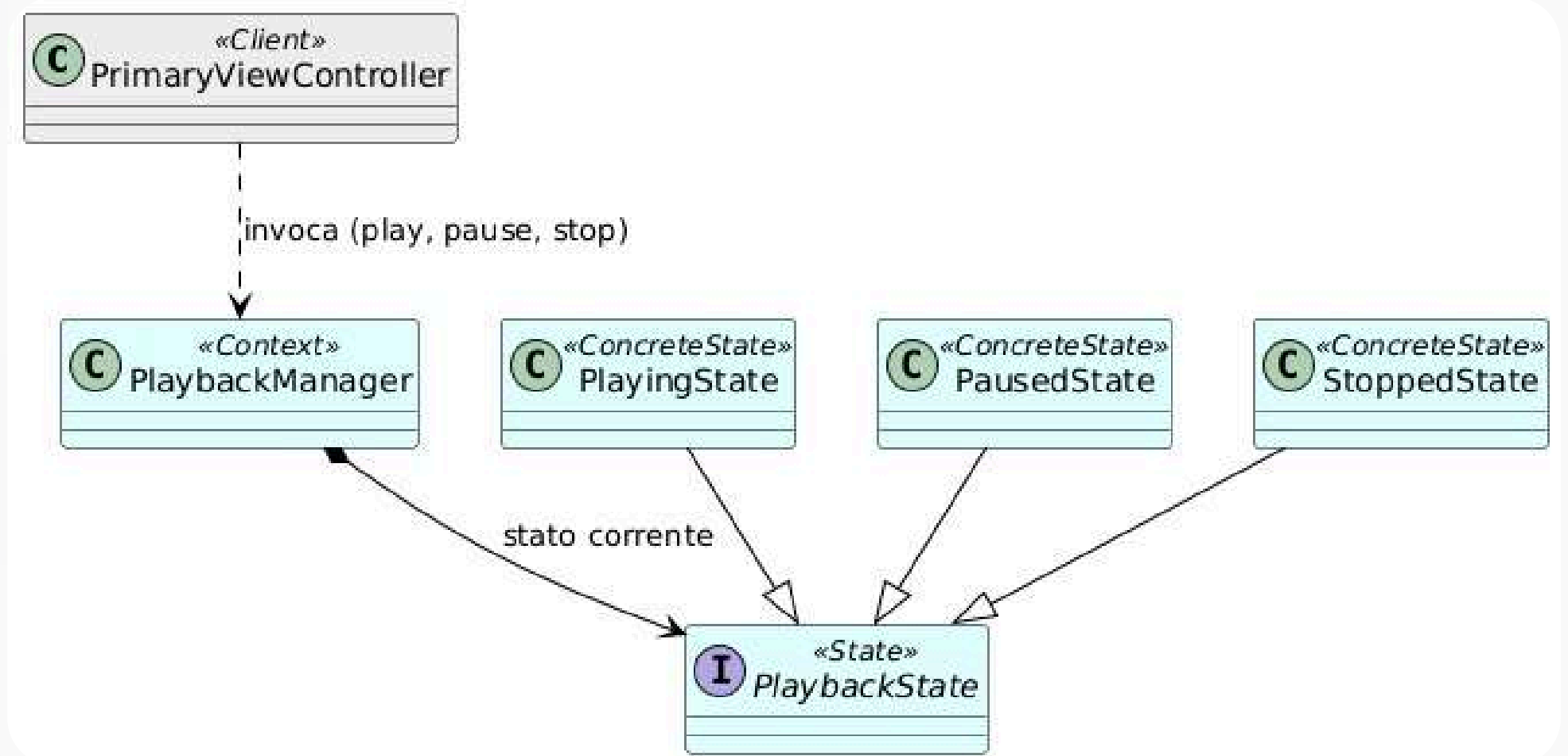
- **Rimuove** i complessi e lunghi blocchi **if-else** o **switch** all'interno del **PlaybackManager** usati per controllare lo stato attuale.

COMPORTAMENTI ISOLATI

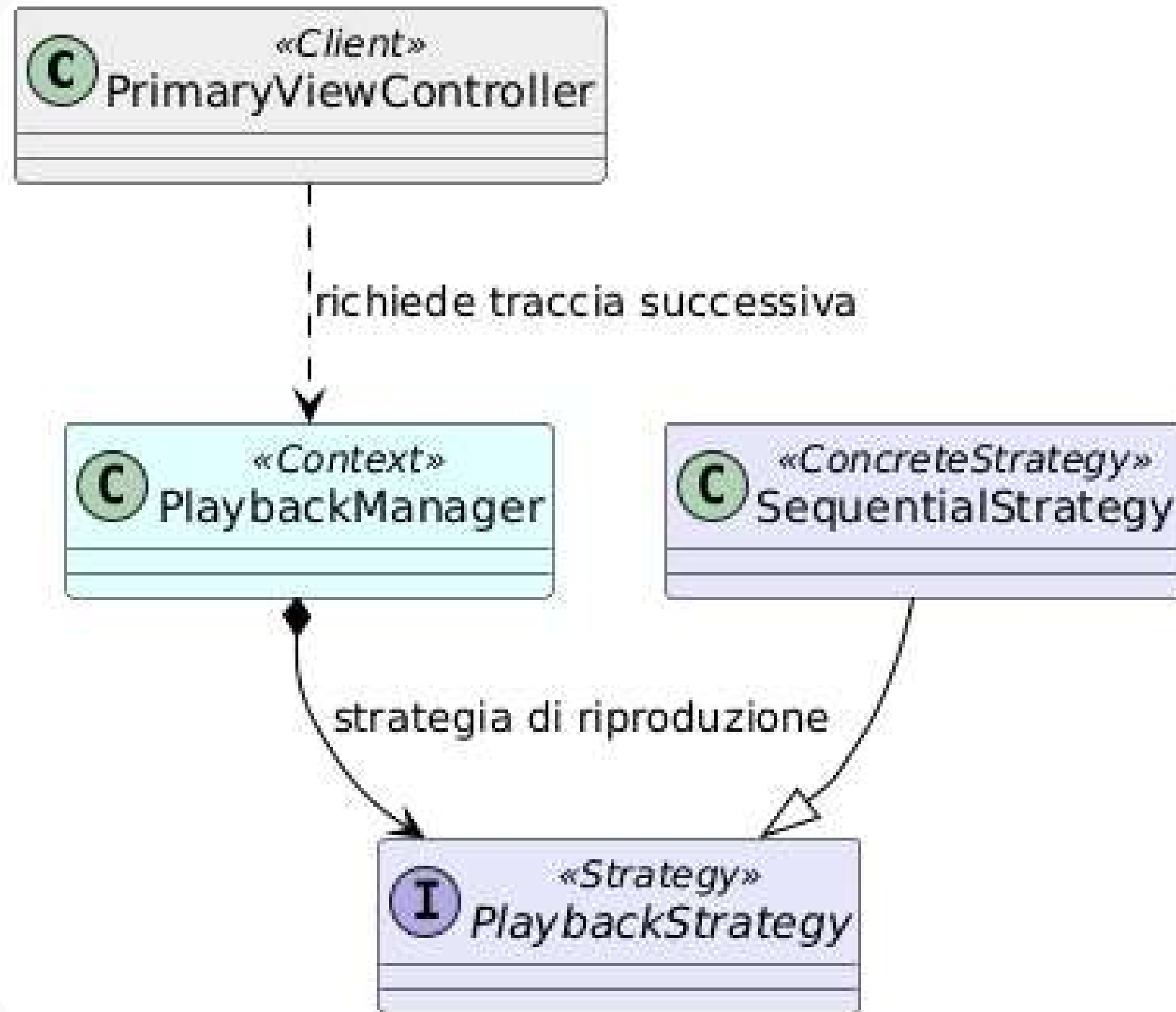
- Ogni stato ha una propria classe dedicata: PlayingState, PausedState e StoppedState. In questo modo, se c'è un problema nella gestione della pausa o dello stop, sappiamo subito dove intervenire.

TRANSIZIONI SICURE

- **Evita bug logici.** Ad esempio, la logica che impedisce di "mettere in pausa una canzone già fermata" è gestita naturalmente dalle regole dello StoppedState



Pattern Strategy - Ordine di riproduzione



ESTENSIBILITÀ - OPEN/CLOSED PRINCIPLE

- Il **PlaybackManager** non contiene direttamente tutta la logica delle modalità di riproduzione. Per aggiungere nuove modalità, come **Shuffle** o **Loop**, basta introdurre nuove classi che implementano **PlaybackStrategy**.

DINAMICITÀ A RUNTIME

- La strategia di riproduzione può essere cambiata durante l'esecuzione del programma, ad esempio tramite un pulsante dell'interfaccia, senza modificare la struttura principale del player.

- In questa fase è stata implementata la **SequentialStrategy**, ma la struttura permette di aggiungere facilmente altre strategie.

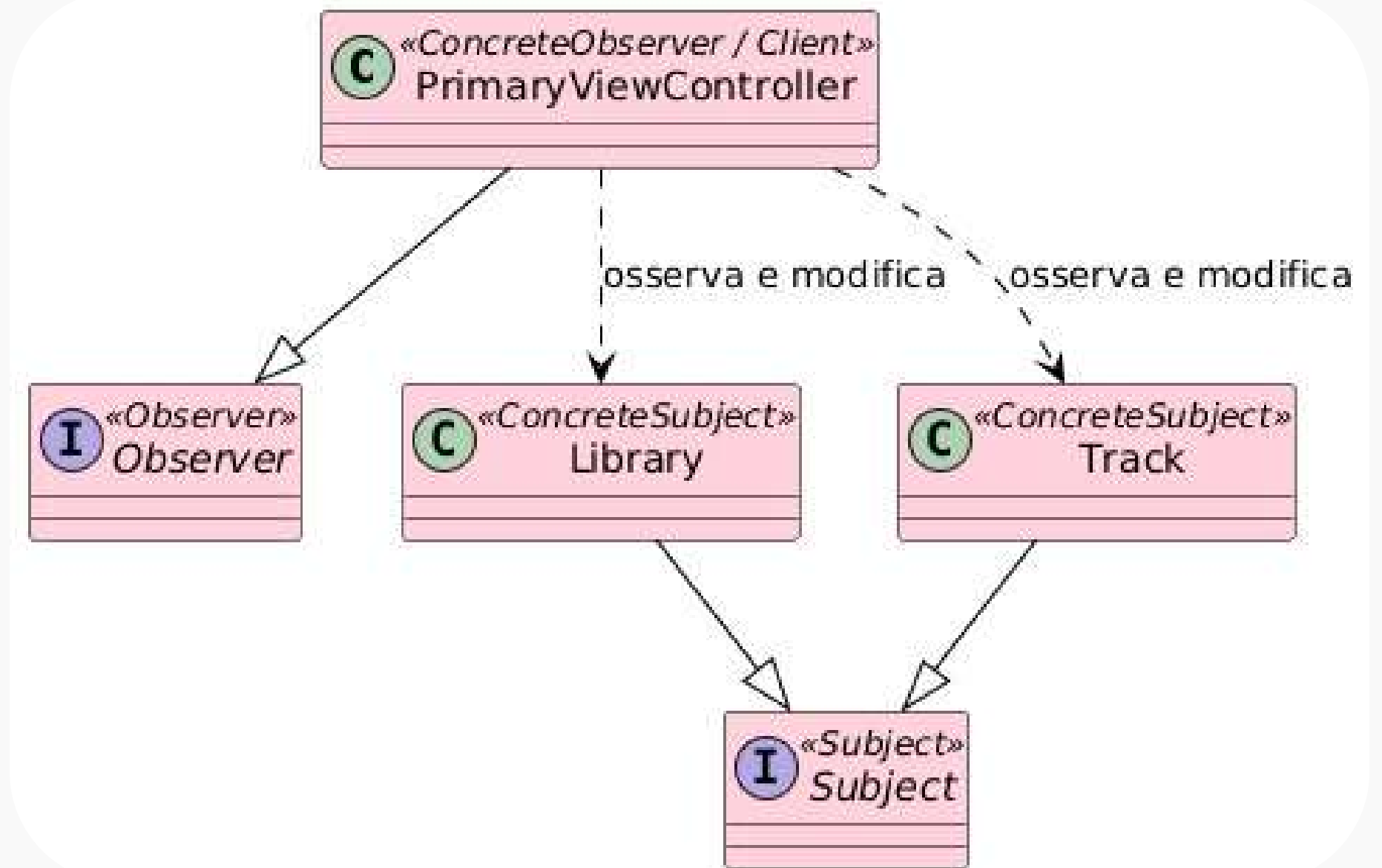
Pattern Observer - Sincronizzazione UI-Modello

DISACCOPPIAMENTO TRA MODELLO E INTERFACCIA

- Il modello dell'applicazione, come Library, non deve conoscere direttamente JavaFX o i componenti grafici. Si limita a notificare gli Observer quando i dati cambiano.

AGGIORNAMENTO AUTOMATICO DELLA UI

- Quando una traccia o una playlist viene aggiunta, modificata o rimossa, il modello invia una notifica. Il Controller, registrato come Observer, aggiorna automaticamente la tabella e gli elementi visivi.
- In questo modo la UI resta allineata ai dati reali senza dover aggiornare manualmente ogni componente dopo ogni operazione.



Trello

Gruppo 1- Progetto Finale SAD

DVMSAAAPDN+1

Burndown Charts for Trello

Story Points

Reports-Beta

Scrum Poker

Estimate Total

Condividi

Traccia progetto e documentazione9

Documentazione

PDF Traccia

Repository GitHub

Board Miro per Mockup

Product Backlog in formato file

Presentazione Pre-Game

Scadenze

Prima consegna: 28 maggio

Scadenze

Seconda consegna: 4 giugno

Scadenze

Terza consegna: 11 giugno

Scadenze

Discussione finale: 18 giugno

+ Aggiungi una scheda

Product Backlog11

High priority

UE2 - Playback Modes

US 2.2 Riproduzione casuale

High priority

UE2 - Playback Modes

US 2.3 - Riproduzione in loop

High priority

UE2 - Playback Modes

US 2.4 - Aggiungere/rimuovere brani durante la riproduzione

High priority

UE3 - User interface and real time ...

US 3.2 - Aggiornamento dell'interfaccia in tempo reale

Medium priority

UE4 - Undo, Analytics and Automa...

US 4.1 - Undo operazione

Medium priority

UE4 - Undo, Analytics and Automa...

US 4.2 - Tracce più riprodotte

Medium priority

UE4 - Undo, Analytics and Automa...

US 4.3 - Playlist più riprodotte

Medium priority

UE4 - Undo, Analytics and Automa...

US 4.4 - Playlist automatiche per genere o anno

Medium priority

UE5 - Tags and filters system

US 5.1 - Assegnazione delle etichette

+ Aggiungi una scheda

Sprint Backlog0

+ Aggiungi una scheda

Task To Do0

+ Aggiungi una scheda

Task In Progress0

+ Aggiungi una scheda

To verify0

+ Aggiungi una scheda

Task complete32

Task 1.1.1 - Logica di business, modello Track e validazione

Task 1.1.2 - Form JavaFX di inserimento e gestione eventi

Task 1.1.3 - Suite di Unit Test JUnit per inserimento brano

Task 1.2.1 - Logica di aggiornamento brano in Library

task 1.2.2 - Schermata di modifica JavaFX e binding dati

Task 1.2.3 - Suite di Unit Test JUnit per modifica brano

Task 1.3.1 - Rimozione da Library e pulizia in cascata

Task 1.3.2 - Dialogo di conferma ed evento di eliminazione

Task 1.3.3 - Suite di Unit Test JUnit per eliminazione

Task 1.4.1 - Dettagli Brano

Task 1.5.1 - Logica Playlist, Composite pattern e validazione

Task 1.5.2- Form JavaFX creazione playlist e eventi

+ Aggiungi una scheda

User Story Done11

High priority

UE1 - Management and rappresent...

US 1.1 - Inserimento brano

High priority

UE1 - Management and rappresent...

US 1.2 - Modifica brano

High priority

UE1 - Management and rappresent...

US - 1.3 Eliminazione brano

High priority

UE1 - Management and rappresent...

US 1.4 - Visualizzazione dettagli brano

High priority

UE1 - Management and rappresent...

US 1.5 Creazione playlist

High priority

UE1 - Management and rappresent...

US 1.6 Modifica playlist

High priority

UE1 - Management and rappresent...

US 1.7 Eliminazione playlist

High priority

UE1 - Management and rappresent...

US 1.8 - Pausa e Avvio del brano

High priority

UE1 - Management and rappresent...

US 1.9 - Saltare un brano nella playlist

High priority

UE2 - Playback Modes

+ Aggiungi una scheda

+ Aggiungi un'altra lista

Inbox

Planner

Bacheca

Cambla bacheche

Trello

Flusso di completamento su Trello

Task To Do → Task In Progress → To Verify → Task Complete → User Story Done

Una task viene considerata completata quando:

- la checklist interna è terminata;
- il codice è stato implementato;
- i test previsti sono stati superati.

Una User Story viene spostata in Done quando:

- tutte le task associate sono completate;
- i criteri di accettazione sono soddisfatti;
- la Definition of Done è rispettata.

The screenshot shows a Trello card in the 'Task To Do' column. The card title is 'Task 1.1.1 - Logica di business, modello Track e validazione'. Below the title, there are buttons for '+ Aggiungi', '🕒 Date', '☑ Checklist', and '📎 Allegato'. Under these buttons, there are sections for 'Membri' (showing a member with a blue circle and 'DN'), 'Etichette' (showing a green label 'Task - Core'), and 'Story Points' (showing a box with '+'). Below this is a 'Descrizione' section with a text input field containing the placeholder 'Aggiungi una descrizione più dettagliata...'. At the bottom, there is a 'Checklist' section with a '0%' progress bar and three items, each with an unchecked checkbox: 'Creare l'entità che rappresenta il singolo brano musicale, dotandola di tutti i metadati richiesti (titolo, autore, durata, genere, anno).', 'Implementare i controlli di validazione per impedire la creazione di brani con dati mancanti, stringhe vuote o valori numerici incoerenti (es. anni o durate negative).', and 'Integrare la logica di inserimento del brano all'interno del gestore centralizzato della libreria'. There is an 'Elimina' button next to the checklist header and an 'Aggiungi un elemento' button at the bottom.

The screenshot shows a Trello board with a column titled 'Task complete' containing 33 items. Three cards are visible: 1. 'Task - Core' with title 'Task 1.1.1 - Logica di business, modello Track e validazione', a green checkmark icon with '3/3', and a blue circle with 'DN'. 2. 'Task - GUI' with title 'Task 1.1.2 - Form JavaFX di inserimento e gestione eventi', a green checkmark icon with '3/3', and a green circle with 'DV'. 3. 'Task - Test' with title 'Task 1.1.3 - Suite di Unit Test JUnit per inserimento brano', a green checkmark icon with '2/2', and a blue circle with 'DN'. A purple arrow points from the 'Task - Test' card down to the 'User Story Done' column in the next screenshot.

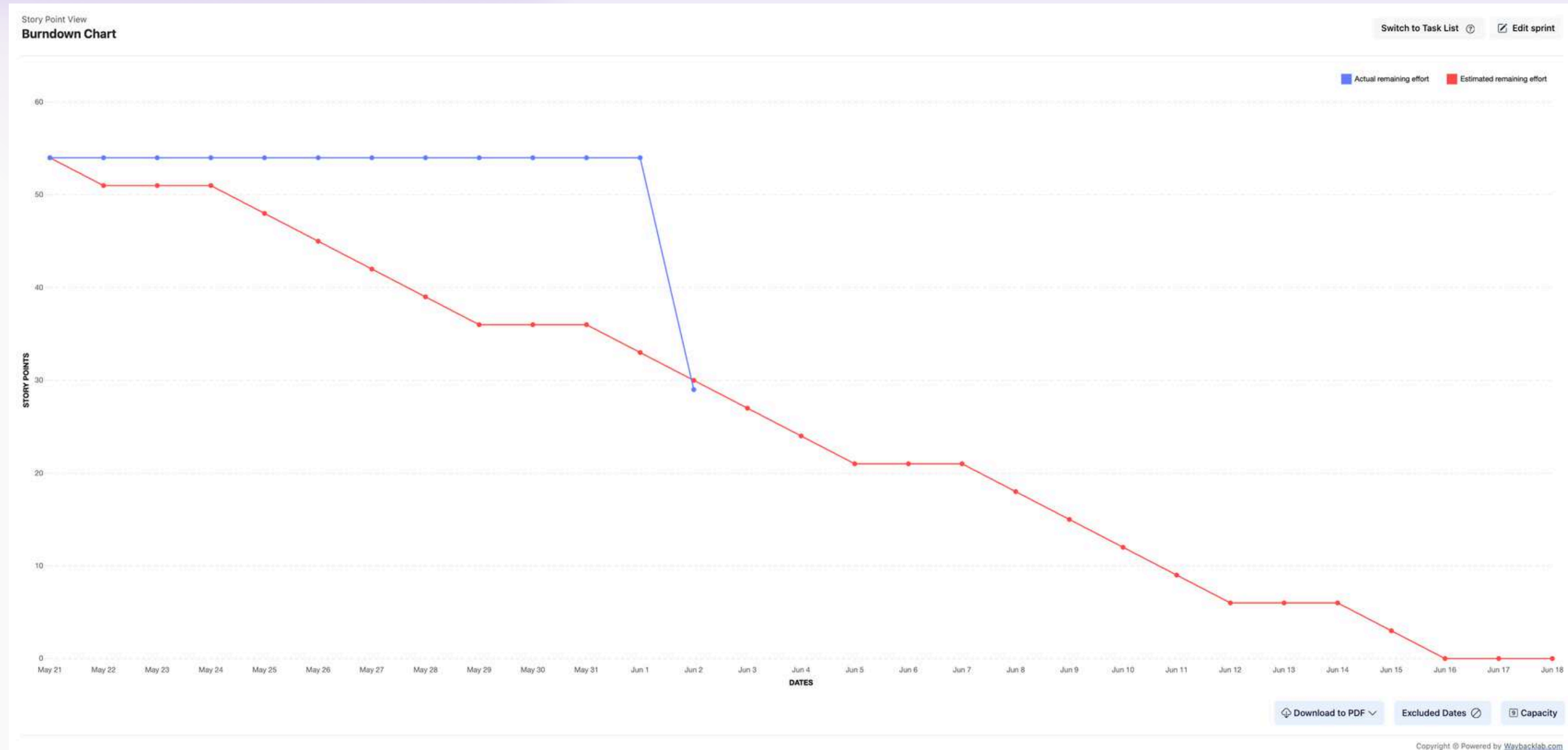
The screenshot shows a Trello board with a column titled 'User Story Done' containing 11 items. One card is visible: 'High priority' with title 'UE1 - Management and rappresent...', a blue label 'UE1 - Management and rappresent...', and subtitle 'US 1.1 - Inserimento brano'. It has a green checkmark icon with '8/8', a green circle with '1', and a trash icon with '1'. A purple arrow points from the 'Task - Test' card in the previous screenshot to this card.

Sprint Backlog

ID User Story	Story Points	Nome Task	Area	Descrizione / Checklist	Stato	Assegnato A
US 1.1 - Inserimento brano	1					
T 1.1.1		core: Logica di business, modello Track e validazione	CORE	- Creare l'entità che rappresenta il singolo brano musicale, dotandola di tutti i metadati richiesti (titolo, autore, durata, genere, anno). - Implementare i controlli di validazione per impedire la creazione di brani con dati mancanti, stringhe vuote o valori numerici incoerenti (es. anni o durate negative). - Integrare la logica di inserimento del brano all'interno del gestore centralizzato della libreria (strutturato come Singleton).	Fatto	DOMENICO NAPPO
T 1.1.2		gui: Form JavaFX di inserimento e gestione eventi	GUI	- Realizzare l'interfaccia grafica per l'inserimento dei dati del brano, prevedendo i campi di input per i metadati e un comando di salvataggio. - Sviluppare il componente di controllo (Controller) per catturare l'azione di salvataggio dell'utente e inoltrare i dati al modulo di business, rispettando la separazione dell'architettura MVC.	Fatto	DEBORA VILLANO
T 1.1.3		test: Suite di Unit Test JUnit per inserimento brano	TEST	- Gestire la visualizzazione degli errori sul modulo grafico, mostrando all'utente un messaggio di avviso nel caso in cui i dati inseriti violino i criteri di validazione. - Scrivere un test automatizzato per verificare che l'inserimento di un brano con dati perfettamente validi vada a buon fine, incrementando correttamente la dimensione della libreria. - Scrivere un test automatizzato per verificare che i tentativi di inserimento con dati non validi vengano intercettati e bloccati, lasciando il sistema inalterato.	Fatto	DOMENICO NAPPO
US 1.2 - Modifica brano	1					
T 1.2.1		core: Logica di aggiornamento brano in Library	CORE	- Implementare la logica per individuare e selezionare in modo univoco il brano da modificare all'interno del catalogo della libreria. - Sviluppare la funzione di aggiornamento per sostituire i vecchi metadati del brano con i nuovi valori inseriti dall'utente. - Applicare le regole di validazione sui dati modificati per evitare il salvataggio di informazioni non coerenti. - Inviare una notifica di aggiornamento tramite il pattern Observer per informare il sistema dell'avvenuta modifica.	Fatto	DOMENICO NAPPO
T 1.2.2		gui: Schermata di modifica JavaFX e binding dati	GUI	- Progettare l'interfaccia grafica dedicata alla modifica del brano, assicurandosi che all'apertura i campi di input mostrino già i valori attuali della traccia selezionata.	Fatto	DOMENICO NAPPO
T 1.2.3		test: Suite di Unit Test JUnit per modifica brano	TEST	- Associare il comando di conferma delle modifiche alla logica di business preposta all'aggiornamento dei dati. - Scrivere un test automatizzato per accertarsi che una modifica valida aggiorni i campi del brano in modo permanente. - Scrivere un test automatizzato per verificare che una modifica non valida venga respinta, lasciando il brano nel suo stato originale.	Fatto	DOMENICO NAPPO
US 1.3 - Eliminazione brano	3					
T 1.3.1		core: Rimozione da Library e pulizia in cascata	CORE	- Sviluppare la logica di rimozione fisica del brano dal catalogo generale della libreria. - Gestione delle dipendenze (Cascata): Implementare il controllo per eliminare automaticamente il brano da tutte le playlist in cui è inserito.	Fatto	DEBORA VILLANO
T 1.3.2		gui: Dialogo di conferma ed evento di eliminazione	GUI	- Notificare l'avvenuta rimozione a tutti i componenti registrati (Observer) per aggiornare contemporaneamente la vista della libreria e delle playlist colpite. - Configurare il controller della UI per intercettare la richiesta di cancellazione del brano selezionato.	Fatto	DEBORA VILLANO
T 1.3.3		test: Suite di Unit Test JUnit per eliminazione	TEST	- Introdurre un messaggio grafico di conferma (finestra di dialogo popup) per chiedere all'utente una convalida esplicita prima di procedere all'eliminazione effettiva. - Scrivere un test automatizzato per verificare che la rimozione di un brano riduca correttamente il conteggio totale dei brani in libreria. - Scrivere un test integrato per assicurarsi che il brano eliminato sparisca automaticamente anche dall'elenco interno di una playlist che lo conteneva. - Verificare tramite test di robustezza che la richiesta di eliminazione di una traccia inesistente non causi errori di sistema o crash dell'applicazione.	Fatto	DEBORA VILLANO
US 1.4 - Dettagli brano	2					
T 1.4.1		gui: Dettagli brano	GUI	- Progettare un'area dedicata nell'interfaccia principale per mostrare le informazioni del brano in modo chiaro e leggibile.	Fatto	VITTORIO POSTIGLIONE
T 1.4.2		test: JUnit e test manuali per visualizzazione dettagli	TEST	- Creare test automatizzati elementari per verificare la corretta lettura e restituzione di tutti i metadati dell'entità brano. - Eseguire un test manuale di verifica dell'interfaccia grafica per accertarsi che il cambio di selezione aggiorni istantaneamente e correttamente i dati a schermo.	Fatto	ARIANNA PALETTA
US 1.5 - Creazione playlist	1					
T 1.5.1		core: Logica Playlist, Composite pattern e validazione	CORE	- Definire la struttura dell'entità Playlist applicando il design pattern Composite: la playlist deve essere un contenitore di elementi riproducibili, potendo così ospitare sia singoli brani che altre playlist annidate. - Introdurre le regole di validazione per il nome della playlist, rifiutando stringhe vuote o prive di caratteri validi.	Fatto	MATTIA SANZARI
T 1.5.2		gui: Form JavaFX creazione playlist e eventi	GUI	- Implementare la logica di controllo per la gestione dei duplicati, decidendo se impedire la creazione di playlist omonime o gestirle tramite identificativi univoci. - Integrare la memorizzazione della nuova playlist nel catalogo generale e inviare la notifica di aggiornamento del modello agli Observer.	Fatto	VITTORIO POSTIGLIONE
T 1.5.3		test: Suite di Unit Test JUnit per creazione playlist	TEST	- Disegnare una schermata o un componente di input per consentire all'utente di digitare il nome della nuova playlist e confermarne la creazione. - Configurare il relativo controller visivo per trasmettere la richiesta al modulo di gestione dati e aggiornare di conseguenza la sezione dell'interfaccia dedicata alle playlist. - Sviluppare un test automatizzato per verificare l'effettivo inserimento e l'aumento del conteggio globale delle playlist a fronte di dati validi. - Sviluppare un test automatizzato per accertarsi che il sistema blocchi la creazione di playlist non conformi ai criteri di validazione (es. nome vuoto).	Fatto	MATTIA SANZARI
US 1.6 - Modifica playlist	2					
T 1.6.1		core: Gestione Composite e logica interna Playlist	CORE	- Sviluppare le funzioni interne alla playlist per gestire l'inserimento e la rimozione di canzoni	Fatto	MATTIA SANZARI
T 1.6.2		gui: Interfaccia di gestione e modifica playlist	GUI	- Realizzare la sezione di interfaccia utente per consultare la composizione di una determinata playlist selezionata.	Fatto	VITTORIO POSTIGLIONE
T 1.6.3		test: Suite di Unit Test JUnit per modifica playlist	TEST	- Integrare i comandi grafici per permettere all'utente di trasferire brani dalla libreria alla playlist o di rimuovere elementi già presenti al suo interno. - Sviluppare test automatizzati per validare le operazioni di inserimento e di rimozione elementi all'interno di una playlist di prova. - Scrivere un test automatizzato specifico per verificare che la playlist ricalcoli e aggiorni sempre in modo corretto la propria durata complessiva a seguito di variazioni del suo contenuto.	Fatto	MATTIA SANZARI
US 1.7 - Eliminazione playlist	1					
T 1.7.1		core: Rimozione playlist e gestione stato Player	CORE	- Sviluppare la funzione per rimuovere in modo definitivo l'entità playlist dal registro del catalogo generale. - Integrità dei brani: Assicurarsi a livello logico che la distruzione del contenitore playlist non influenzi o cancelli i brani musicali in esso contenuti, che devono restare intatti nella libreria globale.	Fatto	DOMENICO NAPPO
T 1.7.2		gui: UI eliminazione playlist	GUI	- Innescare la notifica Observer per aggiornare i componenti visivi.	Fatto	VITTORIO POSTIGLIONE
T 1.7.3		test: Suite di Unit Test JUnit per modifica playlist	TEST	- Collegare l'azione di eliminazione grafica a una finestra di dialogo di sicurezza che richieda la conferma dell'utente prima di inoltrare la richiesta. - Scrivere un test automatizzato per certificare la corretta diminuzione del numero di playlist totali. - Scrivere un test automatizzato per assicurarsi che, in seguito all'eliminazione della playlist, i relativi brani continuino a essere presenti nel catalogo della libreria.	Fatto	MATTIA SANZARI
US 1.8 - Pausa e avvio brano	3					
T 1.8.1		core: State Pattern per la riproduzione (PlaybackState)	CORE	- Progettare la struttura della riproduzione simulata applicando il design pattern State: definire un'interfaccia comune per gli stati del lettore musicale e implementare i singoli stati concreti (In Riproduzione, In Pausa, Interrotto). - Configurare la classe di contesto del riproduttore musicale per ospitare lo stato corrente e gestire le transizioni fluide da uno stato all'altro.	Fatto	ARIANNA PALETTA
T 1.8.2		gui: Integrazione controlli Player JavaFX	GUI	- Sviluppare le funzioni principali di avvio, sospensione e interruzione della riproduzione, delegando il comportamento specifico alle classi di stato attive - inserire controlli di sicurezza che impediscano l'avvio se non è caricato alcun contenuto.	Fatto	ARIANNA PALETTA
T 1.8.3		test: Suite di Unit Test JUnit per State Pattern	TEST	- Implementare nell'interfaccia utente i pulsanti di controllo standard (Play, Pause, Stop) e connetterli al modulo di gestione della riproduzione. - Registrare la UI come ascoltatore dei cambi di stato della riproduzione, in modo che i pulsanti grafici e gli indicatori visivi si abilitino, disabilitino o cambino aspetto in modo coerente con lo stato del lettore. - Scrivere test automatizzati per validare le transizioni di stato del riproduttore (es. verificare il corretto passaggio dallo stato di pausa a quello di riproduzione alla pressione del comando Play). - Verificare tramite test che l'invocazione di comandi di riproduzione in assenza di file caricati venga intercettata senza causare errori anomali.	Fatto	DEBORA VILLANO
US 1.9 - Saltare un brano	3					
T 1.9.1		core: Logica di skip (next/previous) nel PlaybackManager	CORE	- Sviluppare le funzioni per avanzare al brano successivo o ritornare a quello precedente, delegando l'algoritmo di selezione alla strategia di riproduzione correntemente configurata. - Gestione dei casi limite (Edge Cases): Gestire a livello logico lo skip all'indietro effettuato sulla prima traccia (provocando il riavvio del brano da zero) e lo skip in avanti effettuato sull'ultima traccia.	Fatto	ARIANNA PALETTA
T 1.9.2		gui/test: Controlli di skip UI e relativi Unit Test	GUI/TEST	- Garantire il corretto aggiornamento del puntatore del brano attivo. - Posizionare e configurare nell'interfaccia grafica i pulsanti per saltare al brano successivo o precedente, connettendoli al controllore visivo. - Sviluppare test automatizzati per assicurarsi che i comandi di skip spostino la riproduzione in avanti e all'indietro rispettando l'ordine degli elementi. - Scrivere test automatizzati specifici per validare il comportamento del sistema in corrispondenza del primo e dell'ultimo elemento della lista.	Fatto	ARIANNA PALETTA
US 2.1 - Riproduzione sequenziale	3					
T 2.1.1		core: Strategy Pattern e SequentialStrategy	CORE	- Progettare l'interfaccia generale per le modalità di riproduzione applicando il design pattern Strategy, definendo i comportamenti per calcolare la traccia iniziale, successiva e precedente. - Sviluppare la strategia concreta per la riproduzione sequenziale, implementando l'algoritmo per l'avanzamento della playlist.	Fatto	ARIANNA PALETTA
T 2.1.2		gui: Gestione avvio riproduzione da UI	GUI	- Consentire al riproduttore di cambiare dinamicamente strategia e impostare la modalità sequenziale come comportamento predefinito all'avvio dell'applicazione. - Gestire il caso limite dello skip in avanti effettuato sull'ultimo brano della coda, provocando l'arresto della riproduzione.	Fatto	ARIANNA PALETTA
T 2.1.3		test: Suite di Unit Test JUnit per SequentialStrategy	TEST	- Implementare la logica di controllo visivo per cui l'interazione diretta dell'utente su un brano o su una playlist provochi il caricamento automatico delle strutture dati nel riproduttore e l'inizio dell'ascolto. - Assicurarsi che l'interfaccia risponda tempestivamente ai cambi automatici di traccia avviati dalla strategia sequenziale, aggiornando scritte e cursore di selezione. - Scrivere test automatizzati per verificare che la strategia sequenziale restituisca i brani nell'esatto ordine di inserimento della lista. - Verificare tramite test che, una volta raggiunta la fine della playlist, la strategia segnali correttamente la conclusione dell'elenco al riproduttore per consentirne l'arresto. - Verificare i test per i casi limite	Fatto	ARIANNA PALETTA
US 3.1 - Interfaccia principale	5					
T 3.1.1		gui: Struttura Layout Home e Liste JavaFX	GUI	- Definire il layout visivo della schermata principale dell'applicazione, organizzando lo spazio in sezioni indipendenti destinate rispettivamente al catalogo dei brani e all'elenco delle playlist. - Integrare componenti grafici idonei alla visualizzazione di elenchi strutturati di dati, curando l'estetica generale e l'allineamento tramite fogli di stile CSS.	Fatto	VITTORIO POSTIGLIONE
T 3.1.2		core/gui: Integrazione Observer e popolazione dati	CORE/GUI	- Progettare la gestione visuale degli stati vuoti, mostrando all'utente indicazioni testuali di cortesia chiare nel caso in cui non siano presenti brani o playlist nel sistema. - Connettere il modulo di controllo grafico alla libreria dati principale per estrarre e visualizzare le informazioni disponibili all'avvio del software.	Fatto	VITTORIO POSTIGLIONE
T 3.1.3		test: JUnit del modello e test manuali UI	TEST	- Registrare l'interfaccia grafica come ascoltatore (Observer) del modello di business della libreria. - Implementare la logica di aggiornamento automatico della UI, programmando la ridefinizione immediata di tutte le liste visive non appena il modello dati invia una notifica di cambiamento. - Sviluppare un test automatizzato sul modello dati per assicurarsi che i metodi di accesso alla libreria esponano le collezioni in modo corretto e coerente. - Effettuare sessioni di test manuale per certificare l'esperienza d'uso complessiva dell'interfaccia, garantendo che le operazioni sui dati si riflettano visivamente all'istante sullo schermo senza anomalie di visualizzazione.	Fatto	DEBORA VILLANO
	25					

Burndown Chart e Velocity

- Durante la prima Sprint il team ha completato tutte le attività pianificate, per un totale di 25 Story Points completati.
- La velocity effettiva della Sprint 1 è quindi pari a 25 SP.
- Il burndown evidenzia una riduzione del lavoro residuo soprattutto nella fase finale della Sprint: per la Sprint successiva sarà importante aggiornare Trello con maggiore continuità, così da avere un tracciamento più graduale e realistico dell'avanzamento.
- Sulla base della velocity rilevata e dell'esperienza acquisita dal team, nella prossima Sprint sarà possibile stimare una capacità leggermente superiore, mantenendo comunque la velocity come previsione empirica e non come obiettivo rigido.



Retrospective Report

STOP	LESS OF	KEEP DOING	MORE OF	START
• Evitare revisioni architetturali non pianificate durante la Sprint, se non strettamente necessarie.	• Ridurre il tempo dedicato ad attività secondarie prima del completamento delle funzionalità core.	• Sessioni di lavoro in presenza con cadenza programmata, efficaci per allineamento e progettazione condivisa • Comunicazione asincrona tramite WhatsApp e Meet per le sessioni a distanza • Definizione dei tasks per tipologia • Rispetto delle scadenze dello sprint nonostante le difficoltà nella fase di progettazione • Uso di Git con distribuzione del lavoro che ha evitato conflitti di merge	• Aggiornamento continuo di Trello durante lo sprint, per riflettere l'avanzamento reale in tempo reale • Code review durante lo sviluppo, non solo a compilazione avvenuta ma anche sulla correttezza logica e strutturale	• Analisi esplicita delle dipendenze tra task durante il planning, per definire un ordine di esecuzione che eviti blocchi • Definizione più netta dei ruoli nel team, per ridurre ambiguità nell'assegnazione delle responsabilità • Confronto sistematico tra stima iniziale degli story point e effort reale a fine sprint, per calibrare meglio le iterazioni future

Velocity reale	25 / 25 SP completati — obiettivo raggiunto per lo Sprint 1
Sfida principale	La definizione dei pattern architetturali interni ha richiesto più iterazioni del previsto durante lo sprint, generando incertezza nel team e comprimendo il tempo disponibile per l'implementazione
Calibrazione SP	La stima degli story point effettuata in fase di pregame ha rivelato alcune imprecisioni: trattandosi di un progetto nuovo, l'assenza di una base di codice preesistente ha reso più complesso del previsto gettare le fondamenta del sistema, portando a una sottostima dell'effort per alcune user story. Altre, al contrario, si sono rivelate meno impegnative del previsto. A partire dallo Sprint 2 si adotterà una revisione delle stime sulla base dell'esperienza acquisita in questa prima iterazione.
Stato al termine	Il nucleo del software funziona e i test previsti sono stati completati. Rimangono alcune incertezze sull'interazione tra componenti, che verranno consolidate negli sprint successivi
Decisione per Sprint 2	Stimare una capacità leggermente superiore, pari a 28 Story Points, mantenendo la velocity come previsione empirica e non come vincolo rigido.

2nd Sprint planning

Sprint Goal

Completare le funzionalità avanzate di riproduzione e migliorare l'esperienza utente.

User Stories Selezionate

Story Points

US 2.2 Riproduzione casuale	3
US 2.3 Riproduzione in loop	3
US 2.4 Aggiungere/rimuovere brani durante la riproduzione	8
US 3.2 Aggiornamento dell'interfaccia in tempo reale	8
US 4.4 Playlist automatiche per genere o anno	3
US 5.1 Assegnazione delle etichette	3
Totale SP:	28

The screenshot shows a Jira Sprint Backlog for a sprint titled "2nd Sprint planning". The backlog contains six items, all marked as "High priority". The items are grouped by their parent user stories (UE2, UE3, UE4, UE5). Each item shows its name, progress (0/8, 0/7, or 0/7), and story points (3 or 8). A red arrow points from the "Story Points" column of the table on the left to the story points in the Jira backlog.

Priority	User Story	Progress	Story Points
High priority	UE2 - Playback Modes	0/8	3
High priority	US 2.2 Riproduzione casuale	0/8	3
High priority	UE2 - Playback Modes	0/8	3
High priority	US 2.3 - Riproduzione in loop	0/8	3
High priority	UE2 - Playback Modes	0/7	8
High priority	US 2.4 - Aggiungere/rimuovere brani durante la riproduzione	0/7	8
High priority	UE3 - User interface and real time ...	0/7	8
High priority	US 3.2 - Aggiornamento dell'interfaccia in tempo reale	0/7	8
Medium priority	UE4 - Undo, Analytics and Automa...	0/7	3
Medium priority	US 4.4 - Playlist automatiche per genere o anno	0/7	3
Medium priority	UE5 - Tags and filters system	0/7	3
Medium priority	US 5.1 - Assegnazione delle etichette	0/7	3

Trello

Gruppo 1- Progetto Finale SAD

DVMSAAAPDN+1

Burndown Charts for Trello

Story Points

Reports-Beta

Scrum Poker

Estimate Total

Condividi

Product Backlog5

Medium priority

UE4 - Undo, Analytics and Automa...

US 4.1 - Undo operazione

0/788

Medium priority

UE4 - Undo, Analytics and Automa...

US 4.2 - Tracce più riprodotte

0/755

Medium priority

UE4 - Undo, Analytics and Automa...

US 4.3 - Playlist più riprodotte

0/755

Medium priority

UE5 - Tags and filters system

US 5.2 - Playlist automatica per tag

0/733

UE6 - Playlist rearrangements

Low priority

US 6.1 - Riordinamento playlist durante la riproduzione

0/788

+ Aggiungi una scheda

Sprint Backlog6

High priority

UE2 - Playback Modes

US 2.2 Riproduzione casuale

0/833

High priority

UE2 - Playback Modes

US 2.3 - Riproduzione in loop

0/833

High priority

UE2 - Playback Modes

US 2.4 - Aggiungere/rimuovere brani durante la riproduzione

0/788

High priority

UE3 - User interface and real time ...

US 3.2 - Aggiornamento dell'interfaccia in tempo reale

0/788

Medium priority

UE4 - Undo, Analytics and Automa...

US 4.4 - Playlist automatiche per genere o anno

0/733

Medium priority

UE5 - Tags and filters system

US 5.1 - Assegnazione delle etichette

+ Aggiungi una scheda

Task To Do13

Task - GUI

Suddivisione primaryView e Controller associato

0/5DN

Task - GUI

Task - Core

Task 2.2.1 - Implementation - Shuffle Playback

0/5

Task - Test

Task 2.2.2 -Test e Validazione Shuffle

0/4

Task - GUI

Task - Core

Task 2.3.1 - Implementazione Riproduzione in Loop

0/5

Task - Test

Task 2.3.2 - Test e Validazione Loop

0/5

Task - GUI

Task - Core

Task 2.4.1 - Implementazione Modifiche Live alla Playlist

0/5

Task - Test

Task 2.4.2 - Test e Validazione

+ Aggiungi una scheda

Task In Progress0

+ Aggiungi una scheda

To verify0

+ Aggiungi una scheda

Task completate33

Task - Configurazione

Task - Setup iniziale del progetto, build system e ambiente di test

3/3

Task - Core

Task 1.1.1 - Logica di business, modello Track e validazione

3/3DN

Task - GUI

Task 1.1.2 - Form JavaFX di inserimento e gestione eventi

3/3DV

Task - Test

Task 1.1.3 - Suite di Unit Test JUnit per inserimento brano

2/2DN

Task - Core

Task 1.2.1 - Logica di aggiornamento brano in Library

4/4DN

Task - GUI

task 1.2.2 - Schermata di modifica JavaFX e binding dati

2/2DN

Task - Test

+ Aggiungi una scheda

User Story Done11

High priority

UE1 - Management and rappresent...

US 1.1 - Inserimento brano

8/811

High priority

UE1 - Management and rappresent...

US 1.2 - Modifica brano

7/711

High priority

UE1 - Management and rappresent...

US - 1.3 Eliminazione brano

7/733

High priority

UE1 - Management and rappresent...

US 1.4 - Visualizzazione dettagli brano

7/722

High priority

UE1 - Management and rappresent...

US 1.5 Creazione playlist

7/711

High priority

UE1 - Management and rappresent...

US 1.6 Modifica playlist

7/722

High priority

+ Aggiungi una scheda

+ Aggiungi un'altra li

Inbox

Planner

Bacheche

Cambia bacheche



Grazie per l'attenzione!

Gruppo 01

0622702428 – Domenico Nappo

IE22700035 – Arianna Paletta

IE22700189 – Vittorio Postiglione

IE22700199 – Mattia Sanzari

IE22700027 – Debora Villano